Technical reference manual

1.0.0

EtherBench

Hardware IO

This task has one primary responsibility: managing all serial I/O from and to the DUT (Device Under Test). This thread is arguably one of the most complex in the system. It must ensure high-speed, non-blocking writes while simultaneously operating in slave or "spy" mode to capture asynchronous incoming data and stream it to the appropriate target.

To meet these strict timing constraints without overwhelming the CPU, the task heavily relies on Direct Memory Access (DMA) for both transmission and reception. Rather than using costly byte-by-byte interrupts or inefficient fixed-interval polling, it leverages STM32 hardware-level frame detection to flush buffers and route packets dynamically.

Responses are broadcasted using a configurable address mask. This publish-subscribe approach ensures that the hardware task can simultaneously route data to the terminal AND to a physical log file (e.g., via FileX).

RTOS Behavior

The Hardware IO task remains in a suspended state, waiting for an Event Flag (rather than a simple semaphore) to resume execution. This wake-up event can be triggered by two primary sources:

- The main Router dispatching a new datagram to the task's input queue.
- Any managed peripheral triggering a hardware interrupt (e.g., Transfer Complete or Idle line detection).

Upon waking up, the thread evaluates the event flags to instantly identify the source and executes the corresponding state machine logic without blocking.

Peripherals

This thread manages a comprehensive suite of hardware interfaces, including:

- 1x CAN bus
- 3x USART
- 1x I2C
- 1x SPI
- 2x DAC
- 2x ADC

All of these are fully available to the user for arbitrary operations. The user can seamlessly transition these digital peripherals into a "Slave" mode. In this configuration, they act as a passive bus spy: sniffing and decoding the data without ever driving the bus lines.

This operational mode switch can be performed on the fly by sending the appropriate control command. There is no software restriction regarding the mode assignment of digital peripherals. However, the analog section remains locked to its native state by design.

Master Mode

When operating in master mode, all transactions are initiated by the user. The task optimizes the transfer methodology based on the payload size:

- **Polling/IT** for very short transfers (minimizing DMA setup overhead).
- **DMA** for larger payloads.

This dynamic optimization is entirely transparent to the user, as the software interface remains identical.

> ⚠ **Master-Slave Electrical Conflicts**
>
> Forcing a peripheral into Master mode while externally driven is prohibited and electrically hazardous. In Master mode, the output lines are actively driven by the MCU. Attempting to force an external voltage on an actively driven output pin can result in permanent hardware damage. However, protective I/O buffers are implemented on the daughter boards to mitigate these risks.

Responses are generated and dispatched as soon as a hardware transfer completes. To prevent task starvation and ensure high responsiveness, the thread employs an asynchronous, non-blocking state machine. If a new command targets a peripheral that is currently busy executing a previous transfer, the request is pushed at the end of it's input queue, allowing the thread to continue servicing other active peripherals.

Slave Mode

When configured in slave mode, the peripheral reacts to external stimuli identically to a standard sensor. This is highly useful for sniffing and decoding bus traffic using the MCU's native hardware parsers, providing an extremely efficient tool for advanced debugging.

In slave mode, data packets are flushed and dispatched upon detecting the end of a transaction, leveraging hardware events:

- **USART:** Idle line detection (RX line high for a full frame).
- **I2C:** STOP condition detected on the bus.
- **SPI:** Rising edge detected on the Chip Select (SSx) pin.
- **CAN:** Complete frame successfully received in the hardware FIFO.

Once captured, these payloads are wrapped into the standardized IPC message format and forwarded to the Router for proper dispatching based on the active address mask.